Introduction

As you go through each phase of the term project, it's important to keep in mind the information presented in the following sections:

- overview
- description of the project structure

Phase 2 of the term project is worth 20% of your overall project grade.

Academic Honesty

You are reminded that all the code that you submit at each phase of the term project must be

- code that you have written yourself since the start of this term (you must not make use of any code that you have written in the past)
- code found in or based on any of the projects that we present to you in lecture during the term, provided you insert a comment into your code to attribute the source

Please ensure that you do not provide other students with access to your code. By doing so, you risk enabling academic misconduct for which you will be held accountable. Typically, the same grade penalty is applied to the student who enabled access to their code, as to the student who used the code.

If you find the project challenging, ask for help from a course instructor or from one of the TAs *assigned to the course this term*. Do not be tempted to copy code from someone else and do not, under any circumstances, allow another student access to your code in any way.

General Requirements

You should commit and push your code to GitHub after completion of each task, if not more frequently. By pushing your code to GitHub you are effectively making a backup of your work which you'll be able to retrieve if anything happens to the code on your laptop.

All code must be appropriately documented:

- each class should have a class-level comment (that appears just before the class declaration) to describe the information that the class represents
- every method must be documented with requires, modifies and effects clauses, as appropriate (note that simple setter and getter methods, and test methods, do not have to be documented)

You code must pass the checkstyle test on AutoTest, otherwise a grade of zero will be assigned.

All of your tests must pass, otherwise a penalty will be applied to your code coverage score.

Main Tasks

In Phase 2 and, indeed, all future phases of the term project, you will be working with the same repository that was provided to you for Phase 1.

Task 1

You should have already pushed your Phase 1 code to GitHub and requested a grade from AutoTest. If you missed the Phase 1 deadline and are still working on it, please complete Phase 1 before attempting Phase 2. It makes no sense to continue with this phase of the term project unless Phase 1 is complete. Having completed Phase 1, please push your work to GitHub and request a grade from AutoTest so that we have a record of the work that you completed during Phase 1.

Task 2

You are probably familiar with the File → Open and File → Save options that are common in desktop applications that we interact with on a daily basis (e.g. word processors and spreadsheets). The process of saving data to a file from where it can be reloaded at a later time is known as data persistence. If we do not save our data (e.g. essay or spreadsheet calculations) to a file before terminating the program, we lose it (unless the application auto-saves it for us).

In this phase of the term project, you will add data persistence to your application.

For example, in the context of a to-do application, you might add the following user stories:

- As a user, I want to be able to save my to-do list to file (if I so choose)
- As a user, I want to be able to be able to load my to-do list from file (if I so choose)

Alternatively, your user stories might be:

- As a user, when I select the quit option from the application menu, I want to be reminded to save my to-do list to file and have the *option* to do so or not.
- As a user, when I start the application, I want to be given the *option* to load my to-do list from file.

Start by adding two user stories to your README.md file

- one that expresses the user having the *option* to save the *entire* state of the application (e.g. the to-do list, recipe list or state of all pieces in a board game) to file
- another that expresses the user having the *option* to reload that state from file and resume exactly where they left off at some earlier time

Note that "having the option" means automatic save/load with no user interaction is insufficient. Your program may autosave and autoload if you wish, but autosave and autoload do not fulfill the Phase 2 requirements.

Commit and push the revised README.md file to GitHub.

Task 3

There are many different ways in which we can perform data persistence. **For your personal project this term, you must store the entire state of your application, as represented in your model package, as JSON data.** We have developed a simple application to illustrate how this is done. Clone the application from GitHub and run it. It allows the user to interact with a workroom to which a number of thingies/doohickies/thingamajigs can be added. Run the application, add a few thingies to your workroom and then select the save option from the menu. Now, from the Project view in IntelliJ, open the data folder and double-click the file `workroom.json`. You will see something like the following:

```
{
    "thingies": [
        {
            "name": "needle",
            "category": "STITCHING"
        },
        {
            "name": "thread",
            "category": "STITCHING"
        },
        {
            "name": "wool",
            "category": "KNITTING"
        },
        {
            "name": "saw",
            "category": "WOODWORK"
        }
    ],
```

```
    "name": "Alex's workroom"
}
```

In this particular case, the work room was named "Alex's workroom" and four thingies were added:

- a needle in the STITCHING category
- thread in the STITCHING category
- wool in the KNITTING category
- saw in the WOODWORK category

If you now quit the application, run it again and print the thingies in the workroom, nothing will be printed. This is because the application does not remember the state of your work room when you last ran it. Now select the option to the load the work room from file. This will construct a new work room from the JSON data stored in the file `workroom.json`. Select the option to print the thingies again and you should now see exactly the items that were in your work room when you previously saved it to file.

Most, but not all, of the code that is needed to achieve this can be found in the `persistence` package. Take time to study it carefully before moving on.

Task 4

There are a number of JSON libraries available in Java. You **must** use the one provided here. You can download the corresponding jar file using this link. Having downloaded this file, drag it to your project's `lib` folder in IntelliJ. Right-click it and select "Add as library..." and then click OK. The documentation for this library is available here. Note that you will likely not need to use anything other than the `JSONObject` and `JSONArray` classes - be sure to take time to familiarize yourself with the methods provided by these classes.

Add a `persistence` package to your application at the same level as the existing `model` and `ui` packages. Most of the code needed to read and write the state of your application to file will be added to this package. You should feel free to model your code on the sample application that we've provided above but, if you do so, please remember to include a citation.

Test and implement the code necessary to realize the user story that loads data from a file. It is recommended that you generate sample data files specifically for this purpose. Take a look at the tests provided with the sample application for some examples and look at the sample JSON files provided in the `./data` directory. Keep in mind that all data files must be stored in the `./data` directory - including those used for testing. Also note that the path to those files must be relative to the root of your application directory. So it will look like:

```
"./data/myFile.json"
```

rather than an absolute path of the form:

```
"/Users/Sammy/Documents/CPSC210/PersonalProject/MyApp/data/myFile.json"
```

Note that if you use absolute paths to your data files, your tests will not run on AutoTest and this will result in a reduced code coverage score.

Task 5

Test and implement the code necessary to realize the user story that saves data to a file. Again, take a look at the tests provided with the sample application for some hints on how you might approach this.

Task 6

If you wish you may add more user stories to your README.md file and realize them in your code you may do so, but this step is optional. Please note that, if you add new user stories, **all data is expected to be persisted,** so that a user can restore **all** aspects of their application.

Grading

When you are ready, commit and push your code to GitHub. Request feedback from AutoTest by making the

following comment on your commit:

```
@210-bot #project
```

As you are all working on individual projects, AutoTest will obviously not give feedback on the correctness of your code. However, it will check that your code adheres to the CPSC 210 coding style and will measure the degree to which your tests cover your code. The code coverage score that you receive from AutoTest will be used, in part, to determine your overall grade for this phase of the term project as follows:

- 20 pts - (code coverage score * 20) - so 100% code coverage yields a score of 20, whereas 0% code coverage yields a score of 0. These points will not be awarded unless a reasonable attempt has been made to realize the two required user stories for this phase of the term project.
- 10 pts - appropriate REQUIRES/MODIFIES/EFFECTS clauses for all methods added during Phase 2
- 70 pts - realization (includes implementation and testing) of two user stories, one for (at the user's option) saving data and another for (at the user's option) loading, that are relevant to the proposed application (35 marks for each)

Note:

- any attempt to "fake" the code coverage score (e.g. calling a method but not using appropriate `assertEquals`, `assertTrue` and/or `assertFalse` statements to verify the expected behaviour) will result in an automatic zero for code coverage
- if any of your tests fails, a penalty will be applied to your code coverage score
- you must get at least 50% on this phase of the term project. If you don't achieve that grade, you will have to continue to work on this phase of the term project and request a regrade at your next lab for a maximum score of 50% on this phase of the term project
- we recommend you check the "General Requirements" section above to make sure that they have been met

During your demo for this phase of the term project, you should be prepared to

- checkout a copy of the code that you pushed to GitHub *prior to the deadline*

< Previous Next >

© All Rights Reserved

edX Edge

edX

About
Affiliates
edX for Business
Open edX
Careers
News

Legal

Terms of Service & Honor Code